

Data Programming in Python | BAIS:6040

Homework 5. Visualization

Instructions

- Fill in the Python statements needed to complete all the questions below.
- Program everything inside this notebook.
- Use only one cell per question.
- Each code cell must be run with no errors.
- Do not contain any unnecessary code in your answer. (Only the final outcome will be regarded as your answer.)
- If the question requests that you store information in a certain variable, use the specific variables name (case sensitive).
- All problems must be solved in a programmable way.
- When you are done, make sure to save your notebook and then click Restart & Run All before submission to create a clean version.

Questions - NumPy

1. To use NumPy, import the NumPy package as np.

```
In [1]: # Your answer here  
import numpy as np
```

2. Set the random seed to 0 and then create a 10 x 10 array `x` of random integers between 1 (inclusive) and 100 (inclusive).

```
In [2]: # Your answer here  
np.random.seed(0)  
x = np.random.randint(1,101, size=(10,10))
```

```
In [3]: # Check your answer here. (Do not make any change to this cell. Just run this cell.  
x
```

```
Out[3]: array([[ 45,  48,  65,  68,  68,  10,  84,  22,  37,  88],
               [ 71,  89,  89,  13,  59,  66,  40,  88,  47,  89],
               [ 82,  38,  26,  78,  73,  10,  21,  81,  70,  80],
               [ 48,  65,  83, 100,  89,  50,  30,  20,  20,  15],
               [ 40,  33,  66,  10,  58,  33,  32,  75,  24,  36],
               [ 76,  56,  29,  35,   1,   1,  37,  54,   6,  39],
               [ 18,  80,   5,  43,  59,  32,   2,  66,  42,  58],
               [ 36,  12,  47,  83,  92,   1,  15, 100,  54,  13],
               [ 43,  85,  76,  69,   7,  69,  48,   4,  77,  53],
               [ 79,  16,  21, 100,  59,  24,  80,  14,  86,  49]])
```

3. Create a 10 x 10 array `y` of integers from 100 (inclusive) to 1 (inclusive), such that the first row has integers from 100 to 91, the second row from 90 to 81, and so on and so forth. Note that order of values matters.

```
In [4]: # Your answer here
y = np.arange(100, 0, -1).reshape(10, 10)
```

```
In [5]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
y
```

```
Out[5]: array([[100,  99,  98,  97,  96,  95,  94,  93,  92,  91],
               [ 90,  89,  88,  87,  86,  85,  84,  83,  82,  81],
               [ 80,  79,  78,  77,  76,  75,  74,  73,  72,  71],
               [ 70,  69,  68,  67,  66,  65,  64,  63,  62,  61],
               [ 60,  59,  58,  57,  56,  55,  54,  53,  52,  51],
               [ 50,  49,  48,  47,  46,  45,  44,  43,  42,  41],
               [ 40,  39,  38,  37,  36,  35,  34,  33,  32,  31],
               [ 30,  29,  28,  27,  26,  25,  24,  23,  22,  21],
               [ 20,  19,  18,  17,  16,  15,  14,  13,  12,  11],
               [ 10,   9,   8,   7,   6,   5,   4,   3,   2,   1]])
```

4. Select all the values that are in the first 3 rows and the last 7 columns of `x` and save the resulting array in `x_top_right`. Select all the values that are in the last 7 rows and the first 3 columns of `y` and save the resulting array in `y_bottom_left`.

```
In [6]: # Your answer here
x_top_right = x[:3, -7:]
y_bottom_left = y[-7:, :3]
```

```
In [7]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
x_top_right
```

```
Out[7]: array([[68, 68, 10, 84, 22, 37, 88],
               [13, 59, 66, 40, 88, 47, 89],
               [78, 73, 10, 21, 81, 70, 80]])
```

```
In [8]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
y_bottom_left
```

```
Out[8]: array([[70, 69, 68],
               [60, 59, 58],
               [50, 49, 48],
               [40, 39, 38],
               [30, 29, 28],
               [20, 19, 18],
               [10,  9,  8]])
```

5. Before we calculate the dot product of `x_top_right` and `y_bottom_left`, we want to confirm that the number of columns of `x_top_right` equals the number of rows of `y_bottom_left`. Compare the two values using the equality operator (`==`) and save the resulting True value in `ans9`. Do not directly put their values (i.e., 7) in the comparison.

```
In [9]: # Your answer here
# Columns of x should equal the rows of y
ans9 = (x_top_right.shape[1] == y_bottom_left.shape[0])
```

```
In [10]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
ans9
```

```
Out[10]: True
```

6. Calculate the dot product of `x_top_right` and `y_bottom_left` and then save the resulting array in the variable `dot_prod`.

```
In [11]: # Your answer here
dot_prod = np.dot(x_top_right, y_bottom_left)
```

```
In [12]: # Check your answer here. (Do not make any change to this cell. Just run this cell.
dot_prod
```

```
Out[12]: array([[14980, 14603, 14226],
                [13820, 13418, 13016],
                [15810, 15397, 14984]])
```

Data Visualization Questions

For questions 1 to 3, you have a dataframe `dfdi` with the features of diamonds.

```
In [13]: # Do not make any change to this cell. Just run this cell.
from seaborn import load_dataset

dfdi = load_dataset("diamonds")
#dfdi = pd.read_csv("diamonds.csv")
dfdi
```

C:\Users\redmo\AppData\Roaming\Python\Python311\site-packages\pandas\core\arrays\masked.py:61: UserWarning: Pandas requires version '1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).

```
from pandas.core import (
```

Out[13]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

1. (3 pts) Plot a box plot that displays the distributions of the 3 columns `x`, `y`, and `z` in `dfdi`. Set the `figsize` to 15 x 7. Customize other arguments, including title, labels, alignment and so on.

In [14]:

```
# Your answer here

import matplotlib.pyplot as plt
import pandas as pd

plt.figure(figsize=(15, 7))

data = [dfdi['x'], dfdi['y'], dfdi['z']]
labels = ['x', 'y', 'z']

# Create the box plot using matplotlib
plt.figure(figsize=(15, 7))
plt.boxplot(data, labels=labels, patch_artist=True)

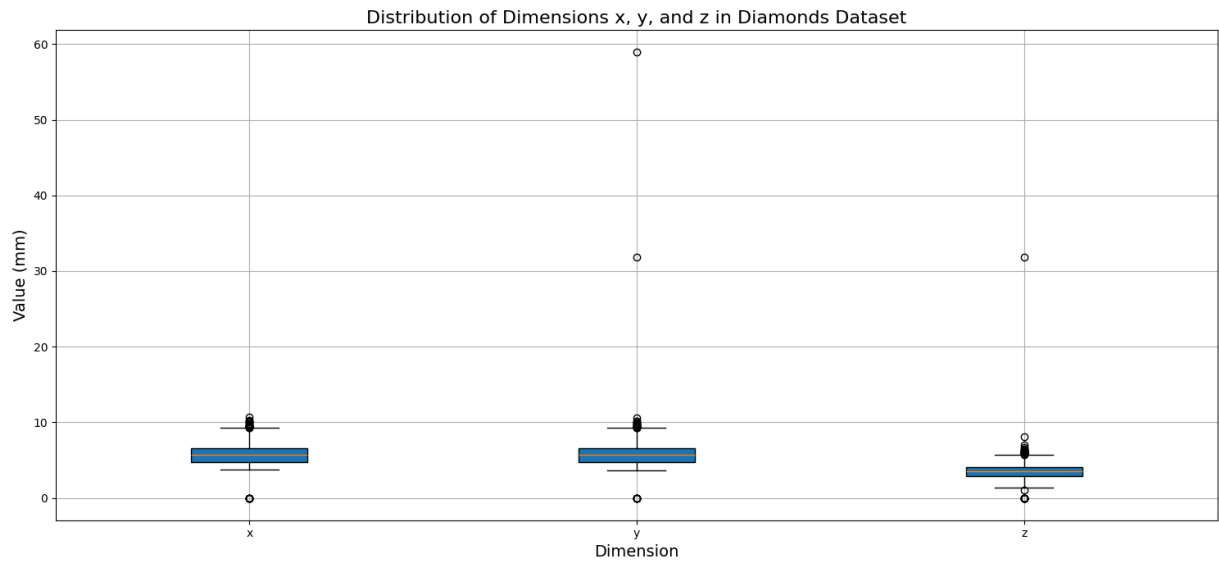
# Customize the plot
plt.title('Distribution of Dimensions x, y, and z in Diamonds Dataset', fontsize=16)
plt.xlabel('Dimension', fontsize=14)
plt.ylabel('Value (mm)', fontsize=14)
plt.grid(True)
plt.tight_layout()
```

```
# Show the plot
plt.show()
```

C:\Users\redmo\AppData\Local\Temp\ipykernel_27396\2236159108.py:14: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
plt.boxplot(data, labels=labels, patch_artist=True)
```

<Figure size 1500x700 with 0 Axes>



2. (4 pts) Plot a histogram that plots the distribution of `price` in `dfdi` with 10 bins. Customize the figsize and other arguments, including title, labels, alignment and so on.

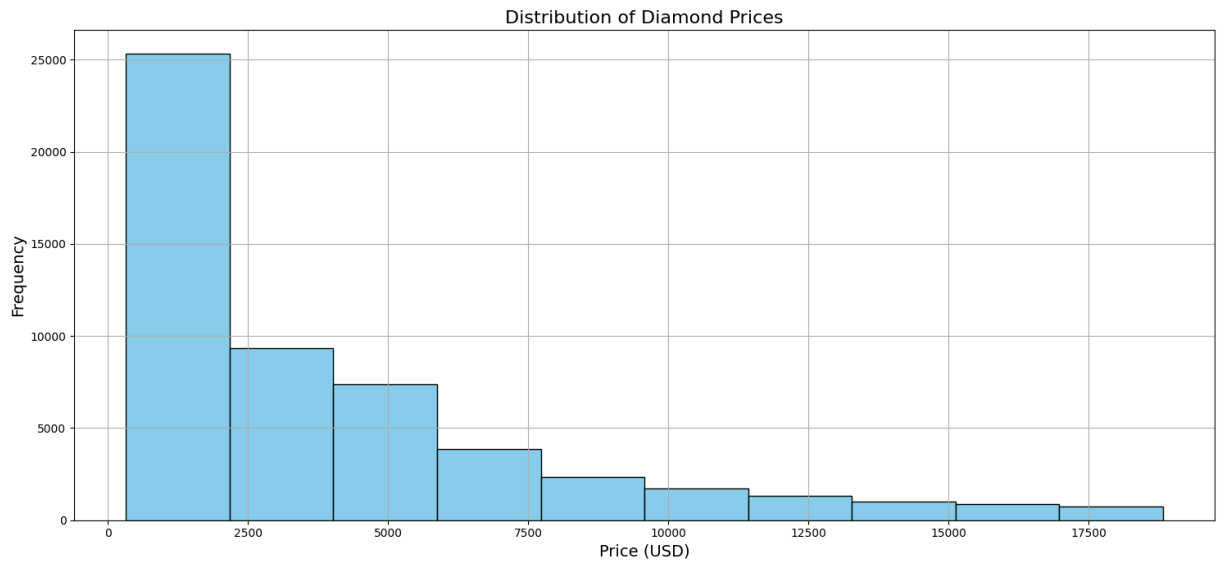
In [15]: *# Your answer here*

```
plt.figure(figsize=(15, 7))

plt.hist(dfdi['price'], bins=10, color='skyblue', edgecolor='black')

# Customize the plot
plt.title('Distribution of Diamond Prices', fontsize=16)
plt.xlabel('Price (USD)', fontsize=14)
plt.ylabel('Frequency', fontsize=14)
plt.grid(True)
plt.tight_layout()

# Show the plot
plt.show()
```



3. (5 pts) There are 7 unique colors in the `color` column of `dfdi`: D, E, F, G, H, I, and J. Plot a pie chart and a bar plot in one figure, two columns in one row. The pie chart displays the distribution(percentage) of the count of each color and the bar plot displays the mean prices for the 7 colors. Set the `figsize` to 14 x 7. Use the `groupby` method to calculate the mean prices for the 7 colors.

```
In [16]: # Your answer here

color_counts = dfdi['color'].value_counts().sort_index()
mean_prices = dfdi.groupby('color')['price'].mean().sort_index()

fig, axes = plt.subplots(1, 2, figsize=(14, 7))

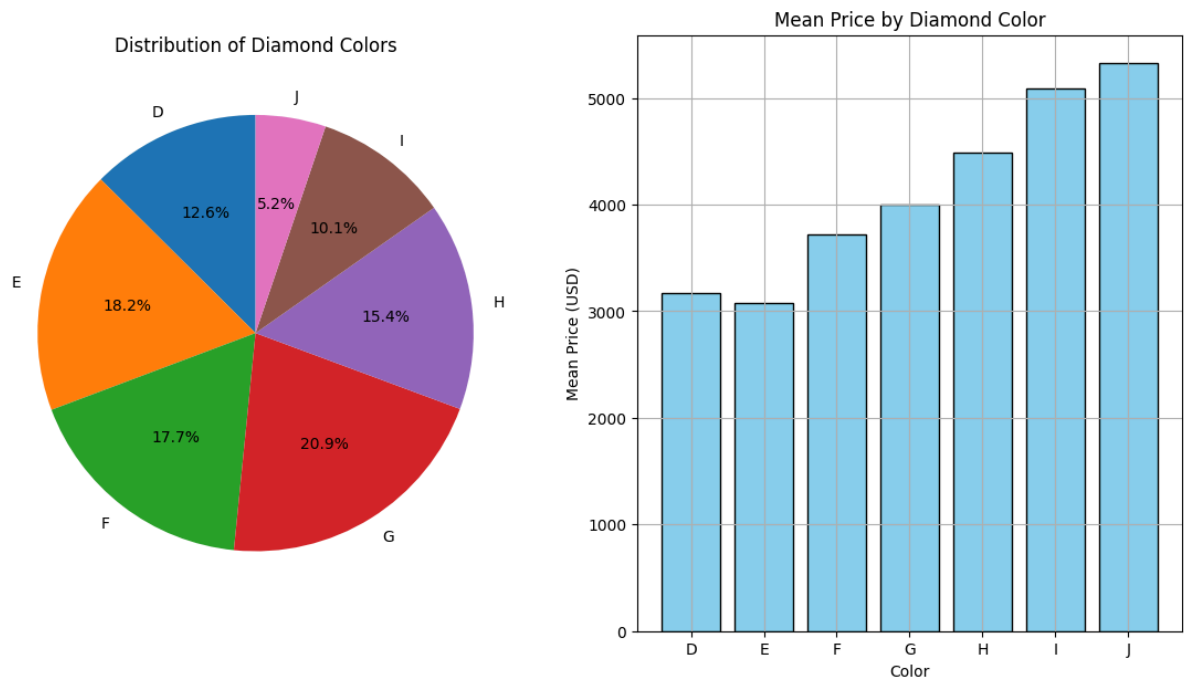
axes[0].pie(color_counts, labels=color_counts.index, autopct='%1.1f%%', startangle=
axes[0].set_title('Distribution of Diamond Colors')

axes[1].bar(mean_prices.index, mean_prices.values, color='skyblue', edgecolor='black')
axes[1].set_title('Mean Price by Diamond Color')
axes[1].set_xlabel('Color')
axes[1].set_ylabel('Mean Price (USD)')
axes[1].grid(True)

plt.show()
```

C:\Users\redmo\AppData\Local\Temp\ipykernel_27396\3304208657.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
mean_prices = dfdi.groupby('color')['price'].mean().sort_index()
```



```
In [19]: # Do not make any change to this cell. Just run this cell.
from seaborn import load_dataset

dft = load_dataset("titanic")
#dfdi = pd.read_csv("titanic.csv")
dft
```

Out[19]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adl
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	
889	1	1	male	26.0	0	0	30.0000	C	First	man	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

891 rows × 15 columns

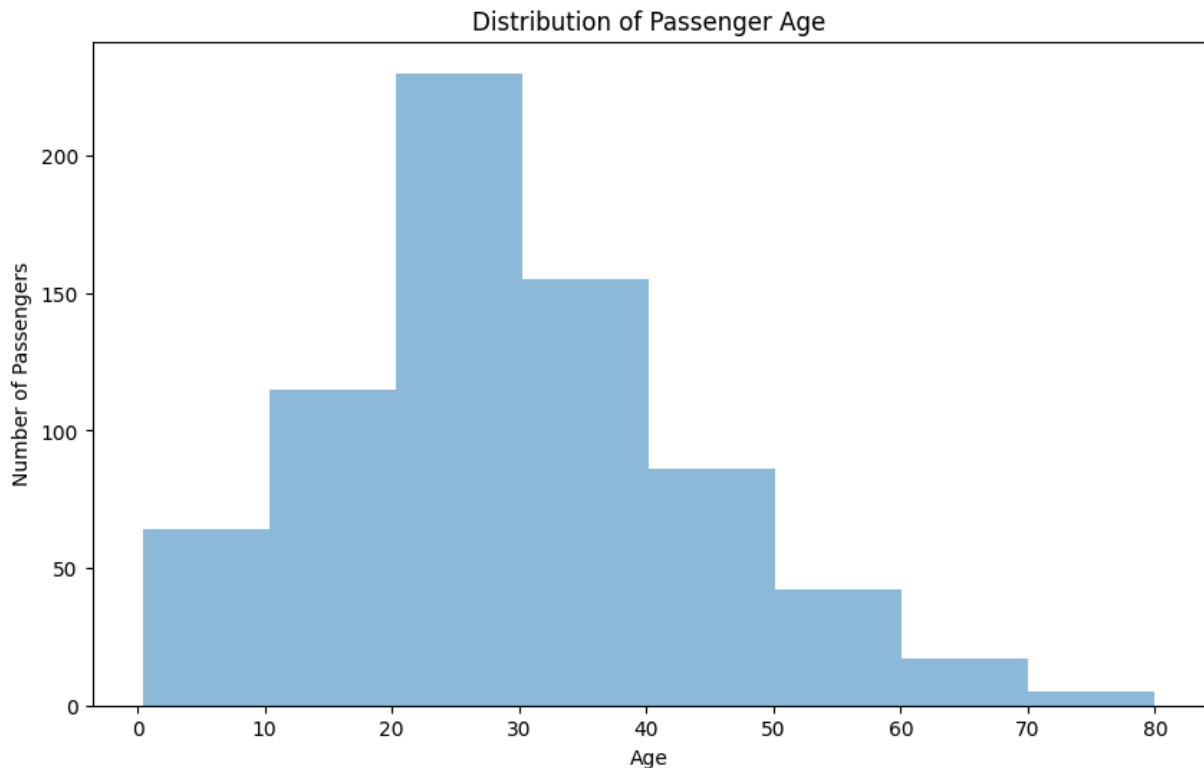


4. Plot a histogram that plots the distribution of `Age` in `dft` with 8 bins. Set the `figsize` to (10, 6), the transparent value `alpha` as 0.5. Set the `xlabel` as "Age", `ylabel` as

"Number of Passengers" add a title "Distribution of Passenger Age".

```
In [21]: # Your answer here
plt.figure(figsize=(10, 6))

# Fill your code below
plt.hist(dft.age, bins=8, alpha=0.5)
plt.xlabel("Age")
plt.ylabel("Number of Passengers")
plt.title("Distribution of Passenger Age")
plt.show()
```

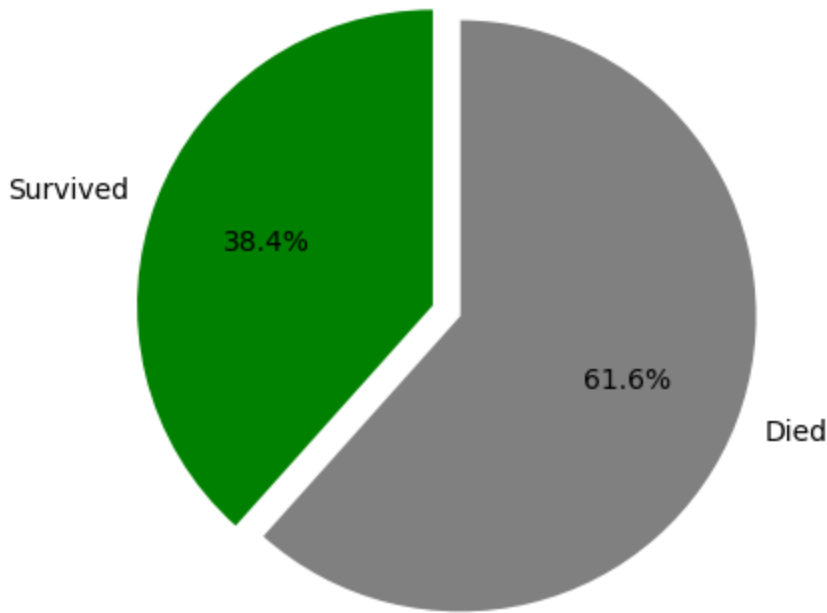


5. Plot a Pie Chart to show the total number of passengers survived or not (died). Set the `startangle` as 90, add the percentage label inside the pie with the format `%.1f%%`, and split the two parts by setting the `explode` as `[0, 0.1]`, set the color as "green" for survived group and "gray" for non-survived group.

```
In [25]: # Your answer here
# Fill your code below
count = [dft.survived[dft.survived==1].sum(), dft.survived[dft.survived==0].shape[0]]

plt.pie(count, labels=['Survived', 'Died'], startangle=90,
        autopct="%.1f%%", explode=[0, 0.1], colors=["green", "gray"] )

# Show the plot
plt.show()
```

6. Plot two pie charts, two columns in one row. Set the `figsize` to 14 x 7. The left pie chart shows the passenger class `Pclass` distribution for the survived passengers; the pie chart on the right shows the `Pclass` distribution for passengers who didn't survive. Add the text labels (given in the `labels` list) and the percentage label inside each pie chart with the format `%.1f%%`, and add a title for each pie chart.

```
In [ ]: pclass_s = dft[dft.survived==1].pclass.value_counts().sort_index()
```

```
In [ ]: pclass_s
```

```
1    136
2     87
3    119
```

```
Name: Pclass, dtype: int64
```

```
In [24]: # Your answer here
# Fill your code below
fig, axs = plt.subplots(1, 2, figsize=(14,7))

ax1 = axs[0]
ax2 = axs[1]

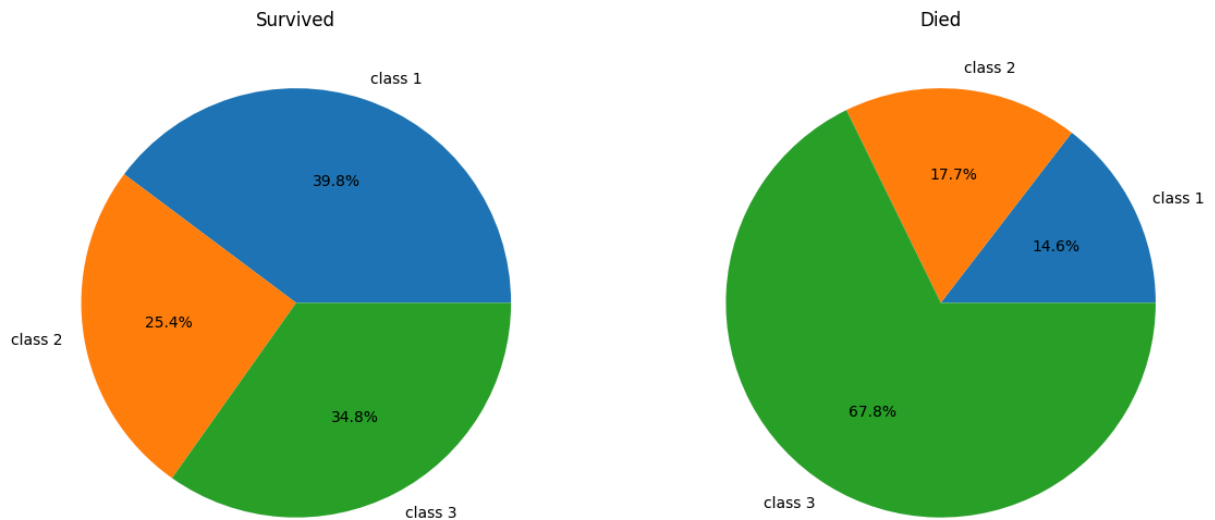
pclass_s = [dft[(dft.survived==1) & (dft.pclass==1)].shape[0], dft[(dft.survived==1)
pclass_d = [dft[(dft.survived==0) & (dft.pclass==1)].shape[0], dft[(dft.survived==0)

labels = ['class 1', 'class 2', 'class 3']

# Pie chart on the left is for passengers who survived
ax1.pie(pclass_s, labels=labels, autopct="%.1f%%")
ax1.set_title("Survived")
```

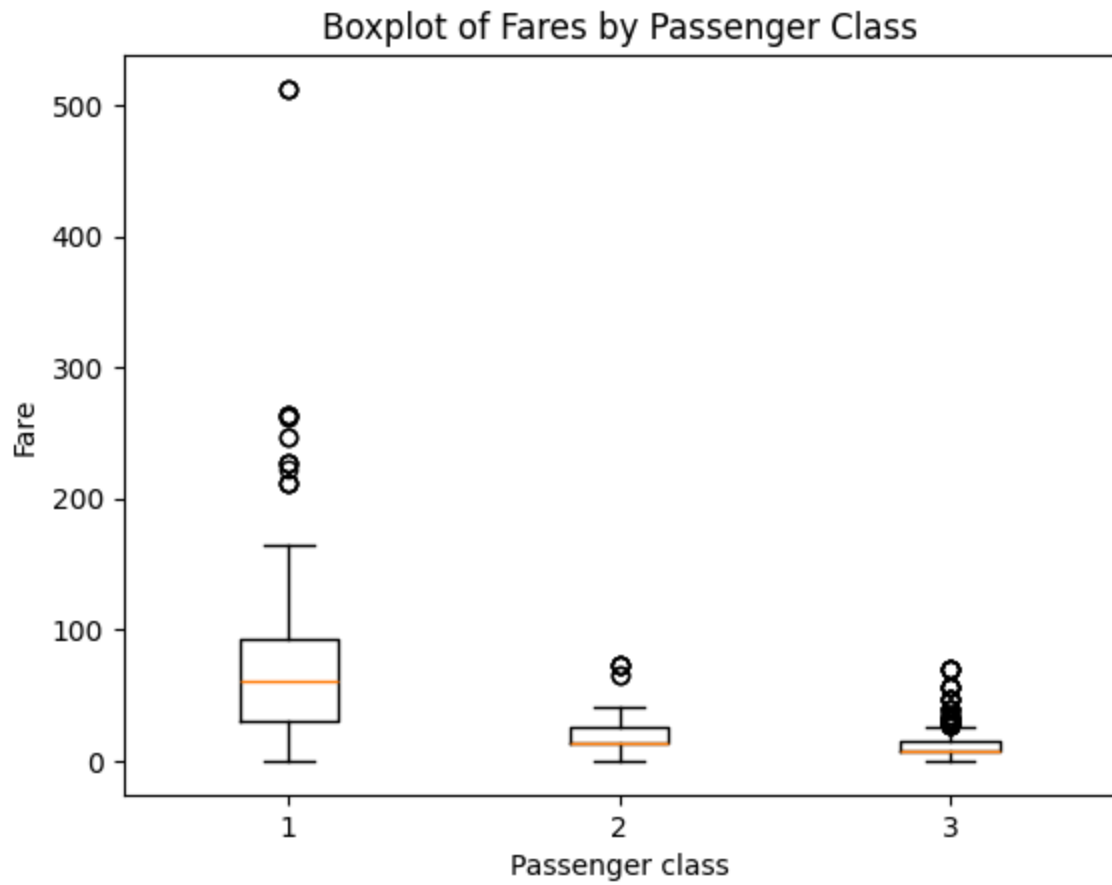
```
# Pie chart on the right for passengers who died
ax2.pie(pclass_d, labels=labels, autopct="%.1f%")
ax2.set_title("Died")

# Show the plot
plt.show()
```



7. Plot a box plot that displays the `Fare` for each `Pclass`. Set the `xlabel` to "Passenger Class", `ylabel` to "Fare", and `title` to "Boxplot of Fares by Passenger Class".

```
In [23]: # Your answer here
plt.boxplot([dft.fare[dft.pclass==1], dft.fare[dft.pclass==2], dft.fare[dft.pclass==3]])
plt.xlabel("Passenger class")
plt.ylabel("Fare")
plt.title("Boxplot of Fares by Passenger Class")
plt.show()
```



In []: